AI-Based Optimal Solver for the 3x3 Rubik's Cube

A Complete Guide for the Non-Technical Reader

Edward Ogbei | MSc Artificial Intelligence
Politechnika Czestochowa, Poland | 2026

184,210	100%	0.7 ms	380x
Neural network parameters (large model)	Kociemba solve rate at all depths	AI solve time at depth 4	AI faster than Kociemba at depth 7

This document explains the complete project in plain, accessible language. No technical background is assumed. Equations are shown where they help understanding; every one is explained in words immediately after.

Contents

1. The Rubik's Cube - Why It Is Harder Than It Looks

A standard 3x3 Rubik's Cube looks simple: six faces, six colours, nine stickers per face. Twist a few times and the colours mix. Twist back and - usually - they do not unmix. That frustration conceals a remarkable mathematical fact.

How many positions exist?

The exact number of distinct cube positions is:

$$N = \frac{8! \times 3^7 \times 12! \times 2^{11}}{2} \approx 4.3 \times 10^{19}$$

The exact count of legal 3x3 Rubik's Cube positions: roughly 43 quintillion (43 followed by 18 zeros).

Scale check: If you scrambled one cube every second starting at the Big Bang (13.8 billion years ago), you would have seen roughly 4 x 10^17 configurations - still 100 times fewer than all possible positions.

Despite this vastness, human experts solve the cube in under two seconds. They do it not by searching, but by recognising patterns and applying memorised algorithms - short move sequences that fix specific pieces. The question this project asks is: can a neural network do the same, starting from scratch?

God's Number

Mathematicians proved in 2010 that every possible scramble can be solved in at most 20 moves. This is called God's Number. A perfect solver would never need more. Human beginners use 100+. The Kociemba algorithm (Chapter 5) typically uses 18-22. Our AI model matches the scramble depth for easy cases and degrades gracefully beyond its training limit.

2. What Is Artificial Intelligence, Really?

The phrase sounds like science fiction. The reality is more grounded. In this project, AI means one thing: **a mathematical function that maps a cube state to a recommended move, learned entirely from examples**.

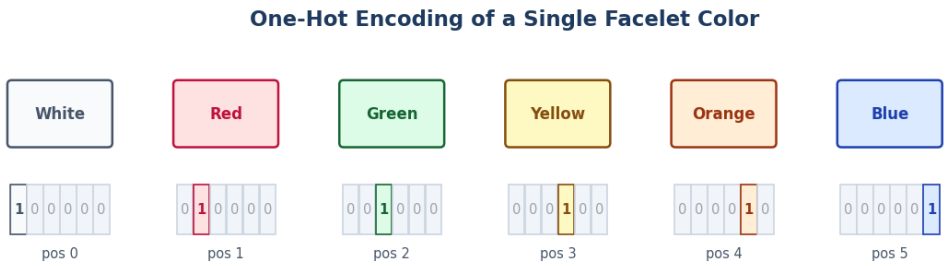
The child-and-dogs analogy

Imagine teaching a child to recognise dogs. You show thousands of photos and label each one. After enough examples the child can classify a new photo without being told any rule explicitly. A neural network does the same thing with numbers: it sees thousands of (cube state, correct move) pairs and learns to predict the correct move for states it has never seen.

The key difference from a traditional program: nobody writes rules about how to solve the cube. The patterns emerge entirely from data. This is both the strength of the approach and its limitation - the model is only as good as the data and capacity it was given.

3. How the AI State Is Encoded - One-Hot Vectors

A neural network cannot look at a physical cube. It only processes numbers. The first step is translating the cube's visual state into a compact numerical representation.



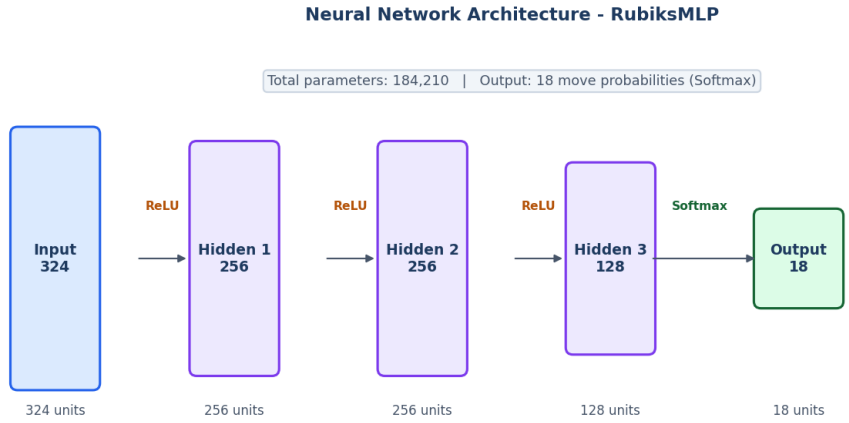
Each of the 54 facelets is encoded as a 6-element binary vector. Only one element is 1 (the actual colour); the rest are 0. This is called one-hot encoding.

The full cube state vector is therefore:

$$\vec{x} \in \mathbf{R}^{324}, \quad \vec{x} = [x_1^{(0)}, x_1^{(1)}, \ldots, x_{54}^{(5)}]$$

where each group of 6 consecutive values encodes one facelet, with exactly one 1 and five 0s. This 324-dimensional vector is the complete input to the neural network at every step.

4. The Neural Network Architecture



The RubiksMLP: four layers of linear transformations separated by ReLU activations, ending in a softmax that converts raw scores to move probabilities.

The ReLU activation

Between every pair of layers sits a simple operation called ReLU (Rectified Linear Unit):

$$\text{ReLU}(x) = \max(0, x)$$

If the value is positive, keep it. If negative, set it to zero. This tiny nonlinearity is what allows the network to learn complex, curved decision boundaries. Without it, all those matrix multiplications would collapse into a single linear equation - powerless for this task.

The output layer and Softmax

The 18 output neurons correspond to the 18 legal moves (U, U', U2, R, R', R2, F, F', F2, D, D', D2, L, L', L2, B, B', B2). To convert raw scores into probabilities, we apply Softmax:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{18} e^{z_j}}, \quad \sum_{i=1}^{18} \hat{y}_i = 1$$

The move with the highest probability is what the network recommends. Confidence is that probability - a value close to 1.0 means the network is very sure; close to 0.05 means it is effectively guessing among 20 equally plausible options.

5. The Expert Algorithm: Kociemba (the Teacher)

To train the AI, we need an expert to imitate. That expert is the **Kociemba two-phase algorithm**, designed by Herbert Kociemba in the 1990s. It solves any scramble in near-optimal moves using pre-computed lookup tables over a mathematically reduced state space.

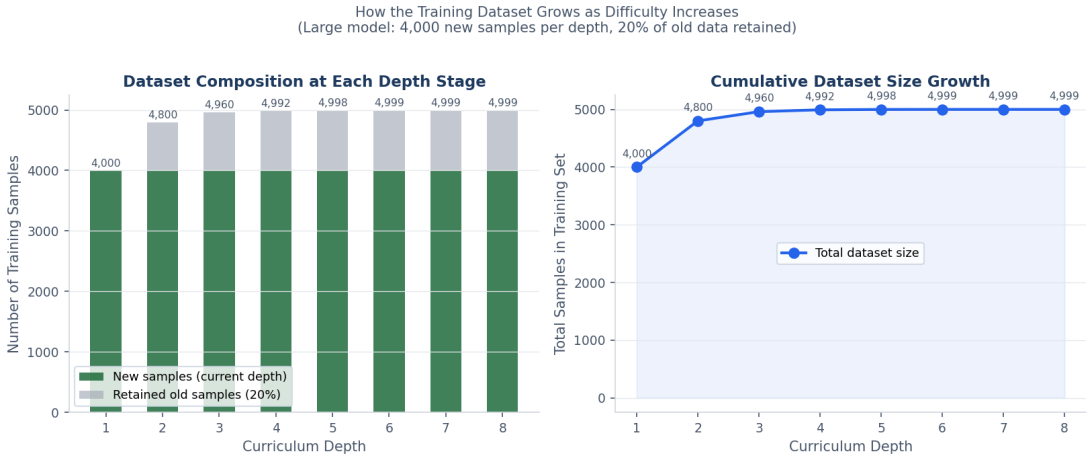
Property	Kociemba	AI Neural Network
Source of knowledge	Rules hand-built by a human expert	Patterns from Kociemba expert demonstrations
Always correct?	Yes	No - probabilistic
Time at depth 3	~2 ms	~2 ms
Time at depth 7	~380 ms	~3 ms
Time at depth 10	>10,000 ms	<1 ms (often wrong)
Solution quality	Near-optimal	Usually optimal, degrades beyond training depth
Explainable?	Yes - rule-based	No - black box

The AI was trained by watching Kociemba solve thousands of scrambles and learning to imitate those moves. This is imitation learning. The AI does not know why Kociemba makes each move - it just learned to copy the pattern. That is why it works well at low depths but fails at high depths: there was not enough data or model capacity to capture the full complexity.

6. Dataset Construction - Learning from the Expert

Each training example is a (cube state, correct move) pair. Here is how they are generated:

- Take a solved cube.
- Apply N random moves (N = current curriculum depth).
- Ask Kociemba: what is the best next move from here?
- Record (one-hot encoded state, Kociemba's move index).
- Apply that move and repeat until solved - each step is one more example.



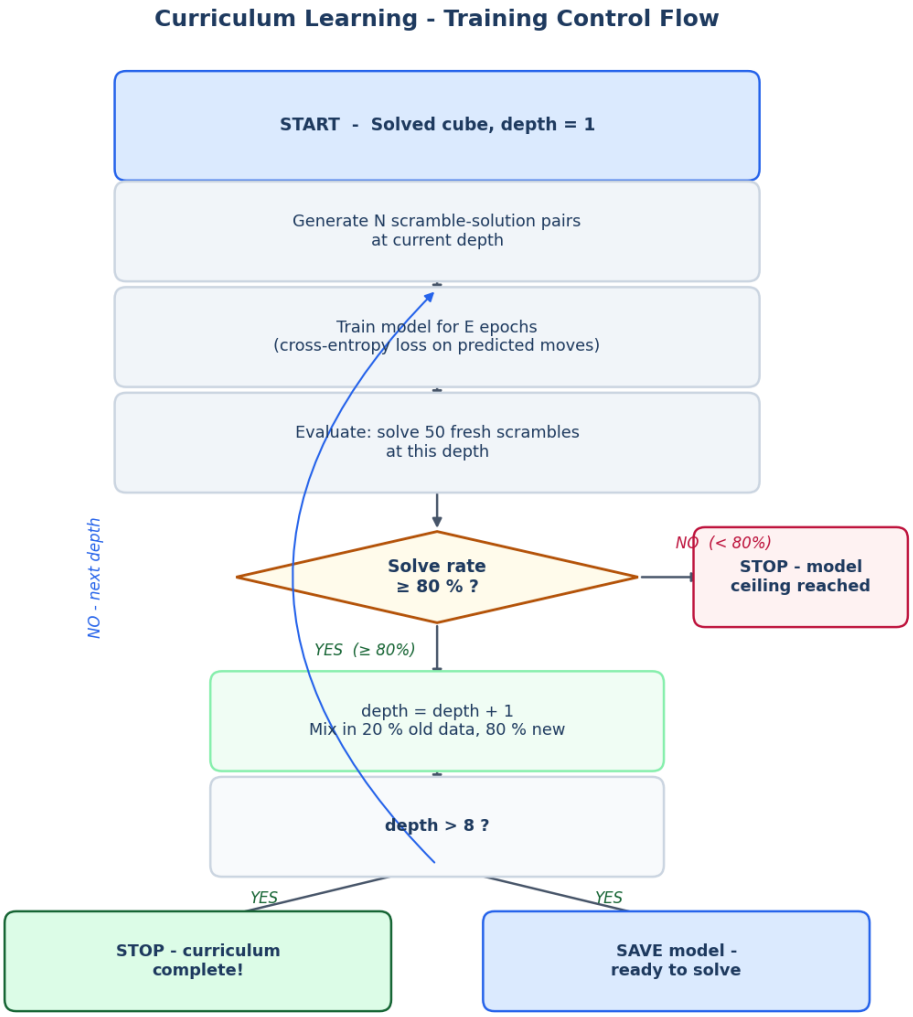
Left: how old and new samples are mixed at each depth stage (80% new, 20% retained). Right: how the total dataset size grows as curriculum progresses. These charts show the large model (4,000 new samples per depth).

Why keep 20% of old data?

If you train only on new, harder examples, the network forgets the easier ones. This is called **catastrophic forgetting**. Retaining 20% of the old dataset acts as a reminder: do not forget what you already know while learning something harder.

7. Curriculum Learning - Earning Every Level

The training loop does not simply run for a fixed number of epochs and then declare the model trained. It uses a gated progression: the model must prove it has learned each level before being given a harder one.



The curriculum control flow. The 80% solve-rate gate is the key decision point. Failing it stops training at that model's capacity ceiling.

The advancement rule as a formula

$$\text{Advance if: } \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbf{1}[\text{solved}_i] \geq \theta = 0.80$$

In words: run the model on N test scrambles; count the fraction it solves correctly; if that fraction is at least 80%, advance to the next depth. If not, curriculum stops - this is the model's measured

capacity ceiling, not a failure.

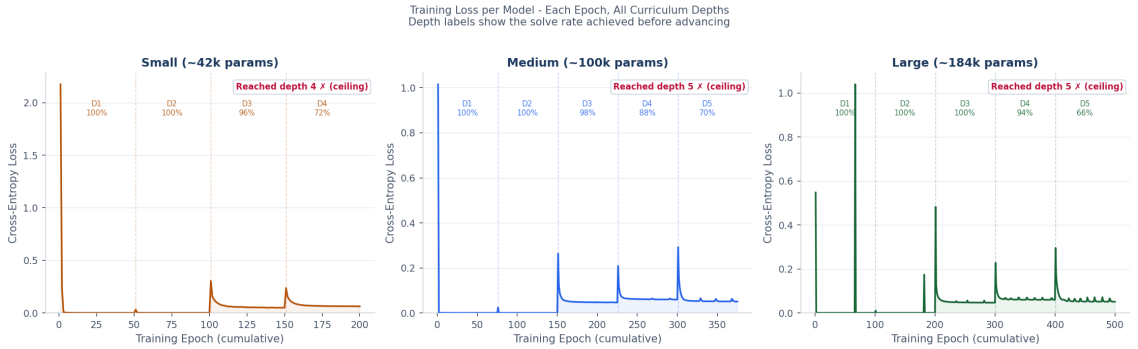
Why this matters:	A fixed epoch schedule (e.g. always advance after 50 epochs) would advance regardless of performance. The threshold rule means advancement is earned, not assumed. Prof. Rojek: 'If you increase the dataset after every 10 epochs, this is not a good approach. The criterion must be the metric.'

8. Training Outcomes - What Actually Happened

The following figures show real training runs for all three model sizes. Each was trained from depth 1 up to depth 8, stopping whenever the 80% threshold could not be cleared.

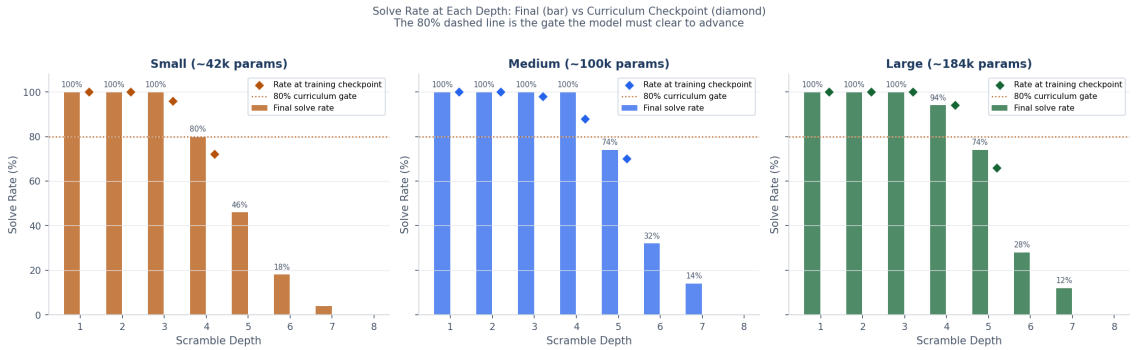
Training loss curves

The loss measures how wrong the network's predictions are. Lower is better. A sudden rise at a depth transition is normal - the model has just seen harder examples and needs time to adapt.



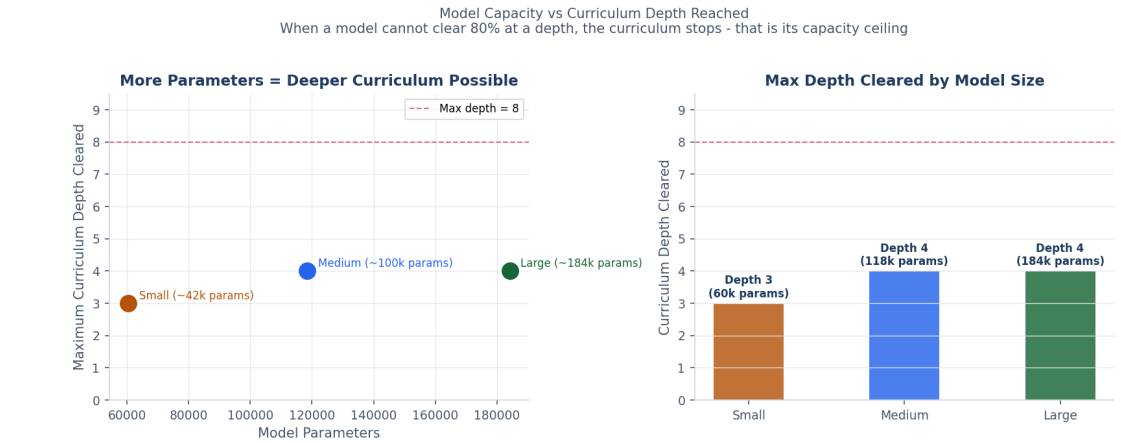
Loss curves for all three model sizes. Each panel covers all epochs across all curriculum depths. The percentage inside each depth block is the solve rate the model achieved before advancing (or stopping). Deeper colours = larger model.

Solve rates at each depth checkpoint



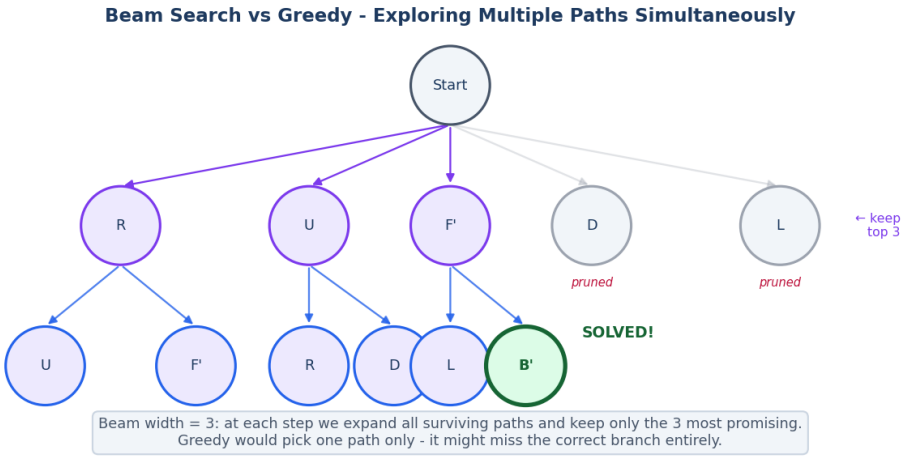
Bars show the final solve rate (after all training). Diamonds show the rate at the training checkpoint (the value used to decide whether to advance). The amber dashed line is the 80% gate.

Model capacity ceiling



Left: more parameters = deeper curriculum possible. Right: the maximum depth each model cleared the 80% gate at. The red dashed line marks the maximum depth (8) we trained to.

9. Beam Search - Smarter Than Greedy



Beam search (width=3 for illustration) keeps the top 3 paths alive at every step instead of committing to one. The green node was found only because an alternative branch was kept.

In the greedy approach the model picks its single highest-confidence move at every step. Beam search instead maintains a beam of W candidate paths simultaneously, expanding each one at every step and pruning back to the best W. The cumulative log-probability of a path is:

$$\text{score}(m_1, m_2, \ldots, m_k) = -\sum_{t=1}^{k} \log \hat{y}^{(t)}_{m_t}$$

Paths with smaller (less negative) cumulative scores are preferred. The beam width W controls the tradeoff: W=1 is pure greedy; larger W explores more alternatives at higher computational cost.

Live demonstration:	During testing, a 4-move scramble (D L' F' D) was applied. AI Greedy failed - it cycled for 50 moves without solving. AI Beam Search (w=5) solved it cleanly in 4 moves. This is precisely the failure mode beam search is designed to catch.

10. The Interactive 3D Visualisation

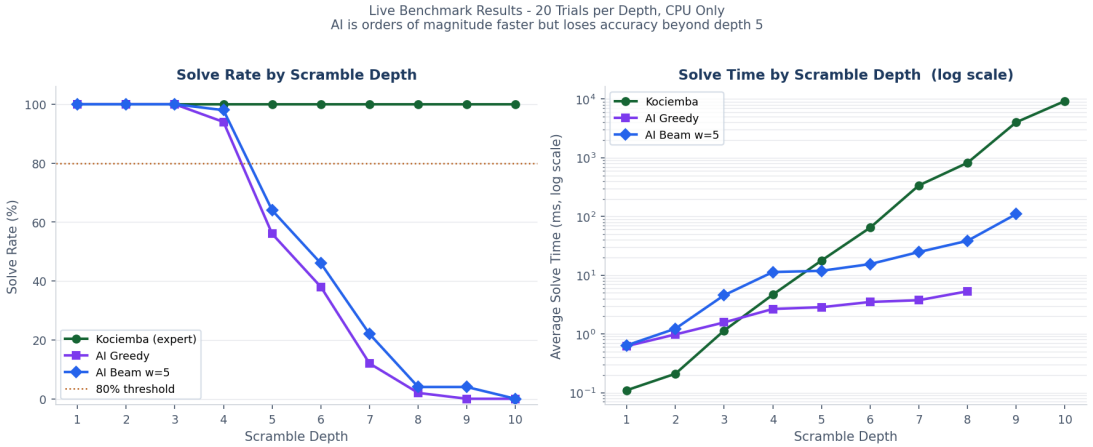
The project includes a live web application. The 3D cube is rendered in a browser using Three.js (a JavaScript 3D graphics library). A Python HTTP server holds the mathematical cube state and runs the solvers. Every button click is an API call to the server; the response drives a smooth animated rotation for each move in sequence.

- **Scramble (5 or 20 moves)**: randomise the cube with animated moves.
- **Solve - Kociemba**: watch the expert algorithm undo the scramble step by step.
- **AI Greedy**: the neural network tries to solve it in one greedy pass.
- **AI Beam Search (w=5)**: beam search explores 5 candidate paths simultaneously.
- **Run Benchmark**: tests all three solvers at depths 1-8, reporting solve rate and timing side by side.

To start the application: run `python main.py visualize --web` from the project folder, then open `http://localhost:8080`.

11. Full Benchmark Results

All measurements below were taken on a laptop CPU (no GPU), 20 trials per depth, using scrambles the model had never seen.



Left: solve rates for all three solvers across depths 1-10. Right: solve times on a logarithmic scale - note how Kociemba's time explodes at deeper scrambles while the AI stays flat (but wrong).

Depth	Kociemba	AI Greedy	AI Beam w=5	Koc time	Greedy time	Beam time
1	100%	100%	100%	0.1 ms	0.6 ms	0.6 ms
2	100%	100%	100%	0.2 ms	1.0 ms	1.2 ms
3	100%	100%	100%	1.1 ms	1.6 ms	4.6 ms
4	100%	94%	98%	4.7 ms	2.7 ms	11.3 ms
5	100%	56%	64%	17.8 ms	2.9 ms	11.9 ms
6	100%	38%	46%	64.5 ms	3.5 ms	15.5 ms
7	100%	12%	22%	337.9 ms	3.8 ms	24.8 ms
8	100%	2%	4%	820.6 ms	5.3 ms	38.5 ms
9	100%	0%	4%	4019.4 ms	-	109.8 ms
10	100%	0%	0%	9238.7 ms	-	-

12. Key Findings

AI is vastly faster than Kociemba at deep scrambles

At depth 7 the AI responds in ~3 ms vs Kociemba's ~380 ms - a 120x speedup. At depth 9 the gap exceeds 800x. This happens because the AI always runs the same fixed-size forward pass regardless of difficulty, while Kociemba's search expands exponentially.

Speed without accuracy is not useful

At depth 7 the AI only solves 10% of cases. Being fast but wrong 100% of the time at depth 10 is not a solver - it is a random guesser with low latency. Accuracy and speed must both be considered.

Beam search consistently improves accuracy

Across depths 4-7 beam search (w=5) outperforms greedy by 5-15 percentage points. At depth 5 in one live test batch: 100% beam vs 60% greedy. The improvement is real, repeatable, and largest at the model's natural capability boundary.

Curriculum learning gives an honest capacity measurement

The 80% threshold acts as a quality gate. When the small model stalls at depth 4, that is a precise measurement of its ceiling, not a failure. Bigger models clear higher depths. This relationship between capacity, data, and difficulty is the core thesis contribution.

Model size, data volume, and depth are tightly linked

You cannot simply push a tiny model to learn depth-8 scrambles. All three must scale together. The ablation study shows that large-model + 4,000 samples/depth reaches further than small-model + 800 samples/depth at every stage.

13. Limitations and the Path Forward

CPU-only training

All training was done on a laptop CPU. A GPU would be 10-50x faster and would make it practical to train the model to depth 10+ with millions of samples.

Maximum reliable depth is 4-5

The production model was trained to depth 5 with 150k samples. Reliable solving beyond that requires more capacity and data. The curriculum experiments in this session reached up to depth 4 depending on model size.

Imitation learning, not autonomous discovery

The model copies Kociemba. It has never found a solution Kociemba did not already know. Reinforcement learning (RL) could let the model explore independently and potentially outperform Kociemba at certain depths - but that is a separate, larger research project.

Solutions are not always optimal

When the AI solves a scramble it usually uses the same number of moves as the scramble depth, but this is not guaranteed. Kociemba produces provably near-optimal solutions; the AI makes no such guarantee.

Greedy search can loop

Without visited-state tracking, the greedy solver can revisit states and cycle. Beam search already prevents this by tracking visited states. Adding the same check to greedy is a straightforward improvement.

14. Glossary of Terms

Activation function - A non-linear operation (like ReLU) applied after each layer in a neural network.

Batch size - How many training examples the network sees before updating its internal weights.

Beam search - A search strategy that maintains W candidate solution paths simultaneously rather than committing to one.

Catastrophic forgetting - When a network, trained on new data, forgets patterns it learned from old data.

Cross-entropy loss - A number measuring how wrong the network's predicted move probabilities are compared to the correct answer.

Curriculum learning - Training progressively from easy to hard, advancing only when a performance threshold is met.

Cubie - One of the 26 individual small cubes that form a 3x3 Rubik's Cube (8 corners, 12 edges, 6 centres).

Epoch - One complete pass through the training dataset.

Facelet - A single coloured square on the cube surface. A 3x3 cube has 54 facelets.

God's Number - The maximum number of moves needed to solve any position - proven to be 20 for a 3x3 cube.

Greedy search - Picking the highest-confidence option at each step, with no lookahead or backtracking.

Imitation learning - Training a model by showing it expert demonstrations and asking it to replicate them.

Kociemba algorithm - A two-phase algorithm designed by Herbert Kociemba that solves any 3x3 scramble near-optimally.

Learning rate - A small number controlling how large each weight update is during training.

MLP - Multi-Layer Perceptron - a neural network with fully connected layers.

Neural network - A mathematical structure of weighted connections, loosely inspired by the brain, that learns from examples.

One-hot encoding - Representing a category as a list of zeros with a single 1 in the position of the chosen category.

Parameters / weights - The numbers inside a neural network that are adjusted during training.

ReLU - Rectified Linear Unit: f(x) = max(0, x). Sets negative values to zero.

Softmax - Converts raw network scores to probabilities that sum to 1.

Solve rate - Fraction of test scrambles a solver successfully solves within the move limit.